

BYTEFORCE

Technical Stack Document

Smart Recruitment Matching Platform

Purpose

This document records the technologies, frameworks, database, languages, APIs, services, tools, and libraries verified directly from the submitted SmartRecruitmentMatchingPlatform source archive.

Project	Smart Recruitment Matching Platform
Team	ByteForce
Document Type	Technical Stack Document
Prepared	11 August 2026

Source verification note

Key details were verified from the .csproj, Program.cs, appsettings.json, controller routes, wwwroot frontend files, EF Core migrations, storage service, and Git metadata contained in the ZIP.

1. Core Technical Stack

The application is a full-stack web system delivered as a single ASP.NET Core project. Static frontend pages are hosted from wwwroot and communicate with the backend REST API.

Category	Technology / Version	How it is used in this project
.NET Version	.NET 8.0 (TargetFramework: net8.0)	Runtime and target framework for the entire ASP.NET Core application.
Frontend	HTML5, CSS3, Vanilla JavaScript	Multi-page responsive web interface stored under wwwroot; no React, Angular, Vue, or other frontend framework is present.
Frontend API Client	Browser Fetch API	JavaScript wrapper sends GET, POST, PUT, PATCH and DELETE requests, attaches JWT bearer tokens, uploads files, and downloads CVs.
Backend	ASP.NET Core Web API on .NET 8	Controller-based REST API with dependency injection, middleware, authorization, DTOs, services, repositories and storage abstractions.
Database	Microsoft SQL Server LocalDB	Configured with Server=(localdb)\MSSQLLocalDB and database name SmartRecruitmentDb.
ORM / Data Access	Entity Framework Core 8.0.11	Code-first entities/configurations, repositories, migrations, relationships, indexes and automatic migration execution at startup.
Primary Languages	C#, JavaScript, HTML, CSS	C# powers backend/business logic; JavaScript powers browser behavior; HTML/CSS define the UI.
Configuration	JSON	appsettings.json, appsettings.Development.json and launchSettings.json configure connection, JWT, storage, logging and local launch settings.

2. Backend Architecture

- Controllers - expose role-protected REST endpoints and validate HTTP requests.
- Services - implement authentication, matching, vacancy, application, profile, CV, notification, contact-request and admin business rules.
- Repositories - isolate Entity Framework Core database queries and persistence.
- DTOs and Entity Models - separate API request/response contracts from persistent domain models.
- Middleware - centralized exception handling returns consistent HTTP error responses.
- Dependency Injection - repositories, services, storage and password hashing are registered with ASP.NET Core DI.

3. APIs and Services Used

The source archive contains an internal REST API rather than integrations with external commercial APIs. The frontend calls these application-owned endpoints through the Fetch API.

API Group	Base Route	Purpose
Authentication	/api/auth	Job seeker/employer registration, login and current authenticated user.
Job Seeker Profile	/api/jobseeker/profile	Profile view/update, dashboard data and skills update.
Employer Profile	/api/employer/profile	Employer profile view, create and update.
Skills	/api/skills	Skill catalog retrieval and creation.
CV Management	/api/cv	CV metadata, upload, secure download and delete.
Vacancy Management	/api/vacancies	Employer vacancy listing, details, create, edit and close actions.
Job Search & Matching	/api/jobs	Job search/details, match score calculation and job application submission.
Applications	/api/applications	Job seeker application history, employer applicant list and application status updates.
Notifications	/api/notifications	List notifications, unread count, mark one read and mark all read.
Contact Requests	/api/contact-requests	Employer contact request workflow, job seeker response and controlled contact detail reveal.
Administration	/api/admin	Admin dashboard, user listing/search and user status management.

Supporting Application Services

- JWT bearer authentication - Microsoft.AspNetCore.Authentication.JwtBearer 8.0.11; issuer, audience, lifetime and signing-key validation.
- Swagger/OpenAPI - Swashbuckle.AspNetCore 6.6.2 with Swagger UI and bearer-token security configuration.
- Password hashing - ASP.NET Core PasswordHasher<User>.
- Custom matching engine - skills 60%, experience 20%, education 10%, location 10%.
- Protected local CV storage - files stored under ProtectedFiles/CVs, outside public wwwroot.
- EF Core startup migration/database seeding plus HTTPS redirection and role-based authorization.

4. Libraries, Tools and Technologies

Technology / Tool	Version / Evidence	Role in the project
JwtBearer	8.0.11	JWT bearer authentication and protected API access.
EF Core SQL Server	8.0.11	SQL Server provider and application data access.
EF Core Design + Tools	8.0.11	Design-time support, migrations and database tooling.
Swashbuckle.AspNetCore	6.6.2	OpenAPI/Swagger generation and interactive Swagger UI.
System.Text.Json	Framework library	API JSON serialization, including string enum values.
Git + GitHub	Repository + GitHub remote	Source control, branch workflow and remote collaboration.
NuGet	PackageReference in .csproj	Dependency/package management for .NET libraries.
ASP.NET Core platform features	Framework	Static files, dependency injection, middleware, HTTPS and authorization.
Local file system storage	ProtectedFiles/CVs	Protected CV storage with generated names and safe-path validation.

5. Frontend Technologies

- HTML5 pages for authentication, job seeker, employer and administrator user journeys.
- CSS3 shared design system (reset, variables, components, layout, responsive/utilities) plus page-specific styles.
- Vanilla JavaScript for API access, authentication/guards, storage, validation and reusable UI helpers. No third-party frontend framework is referenced.

6. Database and Storage Details

Database engine: Microsoft SQL Server LocalDB

Database name: SmartRecruitmentDb

Data access pattern: Entity Framework Core Code First + repository layer

Schema evolution: EF Core migrations (InitialCreate through ContactRequests migrations)

CV storage: Local protected folder; allowed extensions .pdf and .docx; configured maximum size 5 MB

Technical stack summary

Smart Recruitment Matching Platform is a .NET 8 full-stack web application using ASP.NET Core Web API, C#, HTML/CSS/vanilla JavaScript, SQL Server LocalDB, Entity Framework Core 8.0.11, JWT authentication, Swagger/OpenAPI, local protected CV storage, Git and GitHub. No external third-party API integration was found in the submitted ZIP.